PASS Data Community Summit

Host redgate

Exploring SQL Server 2025's Practical Benefits for T-SQL Developers

Andy Yun

He / Him

Field Solutions Architect

Everpure *(Pure Storage)*



Andy Yun

He/Him

I have 80 years of experience (...in dog years)





Field Solutions Architect

Everpure *(formerly Pure Storage)*

Sebastian
"Senior" Developer,
started Monday

- Working w. SQL Server since 2001
- Speaking since 2014
- Redgate Community Ambassador
- Former Microsoft MVP, Data Platform
- Foodie, dog lover, aspiring guitarist, Deborah's personal chef

sqlbek@gmail.com

github.com/sqlbek

sqlbek.wordpress.com

Welcome aboard Sebastian! How're you enjoying your 1st week?





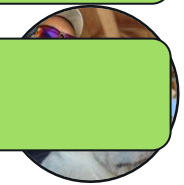
[Sebastian is typing...]

about how you just upgraded everything to SQL

[Sebastian is typing MORE...]

Uhh, ye

[Sebastian is tap, type, tap, tap...]



[Sebastian is STILL typing...]

[Sebastian is typing, typing, typing...]

Pull Request #1: “Minor Changes”



Minor Changes #1

sebastian wants to merge into `main`

Open

487 files changed

Description: *“pls approve and deploy. don’t worry all it works”*



SQL Server 2025 has TONS of awesome new features, let’s use ‘em ALL!!!

Tired of AI…?

- What about SQL Server 2025's T-SQL enhancements?
- Today's Agenda:

All T-SQL & Engine



Tag Definitions

UPGRADE	Works once you're on 2025
COMPAT 170	Change compat level
SWITCH	Off until someone turns it on
PREVIEW	Gated, dev and test only

Chapter 1

“I Found This Code on Stack Overflow”

Say goodbye to janky workarounds!



The Phone Number Validator

Pre-2025

- Nested REPLACE()… REPLACE()… REPLACE()…
- Or… PATINDEX and LIKE with bracket gymnastics
- Or… a CLR assembly nobody remembers deploying

```
SELECT CustomerID, Phone
FROM dbo.Customers
WHERE LEN(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(
        Phone, '(', ''), ')', ''), '-', ''), '.', ''), ' ', '')) = 10
  AND REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(
        Phone, '(', ''), ')', ''), '-', ''), '.', ''), ' ', '')
        NOT LIKE '%[^0-9]%';
```

Real RegEx! Seven New Functions!

- REGEXP_LIKE(): does it match?
- REGEXP_REPLACE(), REGEXP_SUBSTR(), REGEXP_INSTR(), REGEXP_COUNT()
- Table Valued Functions: REGEXP_MATCHES() and REGEXP_SPLIT_TO_TABLE()
- Works in CHECK constraints

```
SELECT CustomerID, Phone
FROM dbo.Customers
WHERE REGEXP_LIKE(Phone, '^\(?\d{3}\)?[-. ]?\d{3}[-. ]?\d{4}$');
```

RegEx: The Fine Print

- Compat 170 required for REGEXP_LIKE() and both Table Valued Functions
- The other scalar functions -> Any compat level!
- SARGable only when the pattern starts with ^
  - Still Scans – Seek support coming

```
SELECT CustomerID, Phone
FROM dbo.Customers
WHERE REGEXP_LIKE(Phone, '^617’);

SELECT CustomerID, Phone
FROM dbo.Customers
WHERE LIKE '617%';
```

PREVIEW

“Jon Smith” vs “John Smyth”

Pre-2025

- SOUNDEX, DIFFERENCE, or “do it in the app”

2025 – Fuzzy Matching!

- EDIT_DISTANCE and EDIT_DISTANCE_SIMILARITY
- JARO_WINKLER_DISTANCE and JARO_WINKLER_SIMILARITY
  - Similarity functions score 0 to 100

```
SELECT DIFFERENCE('Jon Smith', 'John Smyth');   -- 0 to 4 score

SELECT EDIT_DISTANCE('Jon Smith', 'John Smyth'),   -- 2025 (preview)
       EDIT_DISTANCE_SIMILARITY('Jon Smith', 'John Smyth'),
       JARO_WINKLER_SIMILARITY('Jon Smith', 'John Smyth');
```

PREVIEW

Fuzzy Matching: Still Fuzzy on GA

- Requires PREVIEW_FEATURES database scoped configuration
- Not recommended for production
- Still preview everywhere, including Azure SQL




```
ALTER DATABASE SCOPED CONFIGURATION
        SET PREVIEW_FEATURES = ON;
```

Honorable Mentions: Part 1

- SUBSTRING: length is now optional
- || and ||= for string concatenation
- CURRENT_DATE returns a date datatype value
- DATEADD accepts bigint
- UNISTR for Unicode escapes



Honorable Mentions: Part 2

Multiply all values of a column together?

- PRODUCT(x)
  - PRODUCT ignores NULLs and supports OVER

Need to BASE64-Encode something?

  - Pre-2025 = XML tricks

- BASE64_ENCODE and BASE64_DECODE
  - BASE64_ENCODE has a URL-safe option

```
SELECT PRODUCT(1 + Rate) FROM dbo.Returns;   -- 2025

SELECT BASE64_ENCODE(CAST('Sebastian' AS varbinary(20)));   -- 2025
```

Demo



Chapter 2

“It Worked on My Machine”

Code you’d write differently now

Literally, Your Machine

- Standard Developer Edition
- Enterprise Developer Edition
- Express: 50 GB databases



Are you REALLY JSON?

Pre-2025

- nvarchar(MAX) pretending to be JSON
- ISJSON() check constraint
- Whole document rewritten to change one value
- Reparsed on every read

```
CREATE TABLE dbo.Orders (OrderID int PRIMARY KEY,
  Doc nvarchar(MAX) NOT NULL CHECK (ISJSON(Doc) = 1));

UPDATE dbo.Orders SET Doc = JSON_MODIFY(Doc, '$.status', 'shipped')
WHERE OrderID = 42;
```

A Real JSON Type

- Native binary type, stored pre-parsed
- Available on every compat level
- modify method updates values in place
- No changes to existing (2016+) T-SQL JSON functions
- ALTER TABLE on existing varchar(max) column

```
CREATE TABLE dbo.Orders (OrderID int PRIMARY KEY,
  Doc JSON NOT NULL);

UPDATE dbo.Orders SET Doc.modify('$.status', 'shipped')
WHERE OrderID = 42;
```

JSON: Build It and Find It

- JSON_CONTAINS () for searching
- JSON_OBJECTAGG() and JSON_ARRAYAGG ()
- RETURNING JSON () gives you the native datatype
- CREATE JSON INDEX: one per JSON column

```
SELECT OrderID, JSON_OBJECTAGG(Sku:Qty RETURNING JSON) AS Lines
FROM dbo.OrderLines GROUP BY OrderID;

CREATE JSON INDEX ix_Orders_Doc ON dbo.Orders (Doc) FOR ('$.status');

SELECT OrderID FROM dbo.Orders
WHERE JSON_CONTAINS(Doc, 'shipped', '$.status') = 1;
```

The Kitchen Sink Search Procedure

- Six optional parameters
- col = @p OR @p IS NULL, six times over
- One cached plan for every combination
- or… OPTION (RECOMPILE)

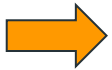
```
CREATE PROCEDURE dbo.SearchCustomers
  @FirstName nvarchar(50) = NULL, @LastName nvarchar(50) = NULL,
  @City nvarchar(50) = NULL  /* ...and three more */
AS SELECT CustomerID, FirstName, LastName FROM dbo.Customers
WHERE (FirstName = @FirstName OR @FirstName IS NULL)
  AND (LastName  = @LastName  OR @LastName  IS NULL)
  AND (City      = @City      OR @City      IS NULL)  -- ...x6
OPTION (RECOMPILE);  -- 'for luck'
```

OPPO: Two Plans Are Better Than One

"Optional Parameter Plan Optimization"

- One statement, multiple plan variants
- Picks at runtime: is the parameter NULL or not?
- Builds on Parameter Sensitive Plan (PSP) from 2022
- On by default at compat 170

```
WHERE LastName = @LastName
   OR @LastName IS NULL
```



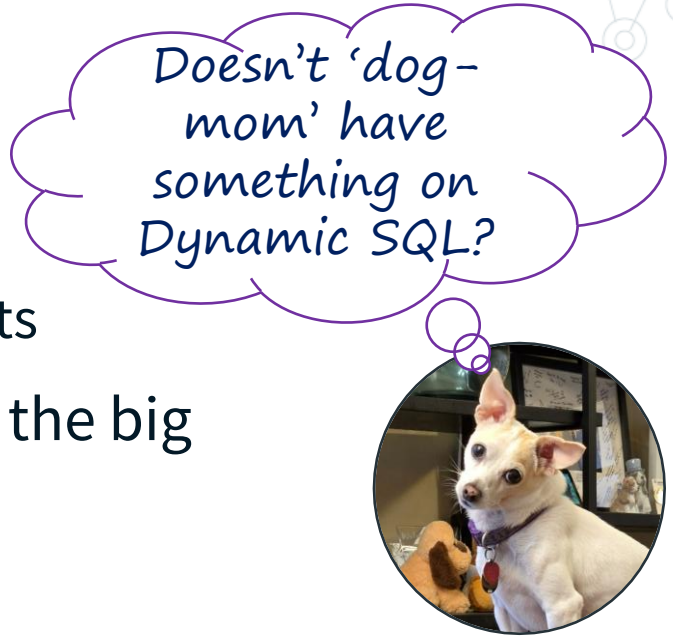
Parameter is NULL: **scan plan**

OR

Parameter has a value: **seek plan**

OPPO: Manage Your Expectations

- Best case: exactly one optional parameter
- Is Skipped …
  - for local variables
  - with OPTION (RECOMPILE)
  - for auto-parameterized statements
- Dynamic SQL is still the answer for the big search proc



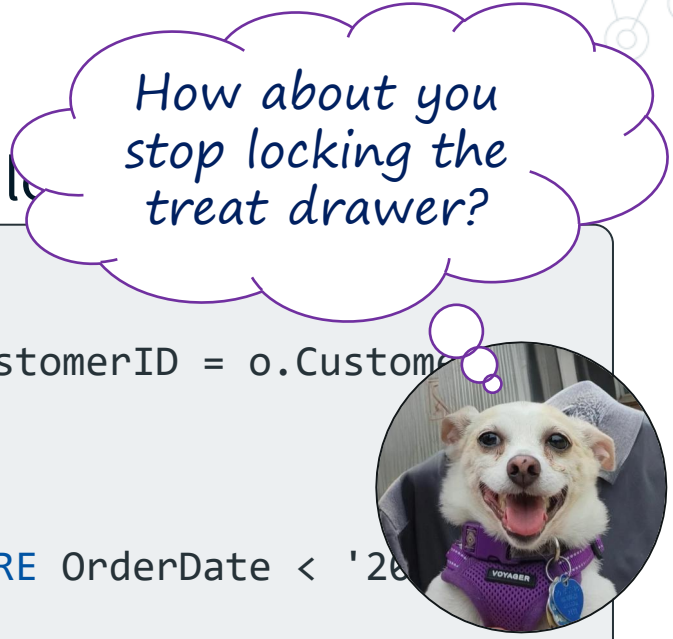
NOLOCK, Everywhere

Pre-2025

- Stop blocking me!
- DELETE in batches
- Lock escalation -> blocking & deadl

```
SELECT o.OrderID, c.CustomerName
FROM dbo.Orders AS o WITH (NOLOCK)
JOIN dbo.Customers AS c WITH (NOLOCK) ON c.CustomerID = o.Custom

WHILE 1 = 1
BEGIN
  DELETE TOP (4999) FROM dbo.OrderHistory WHERE OrderDate < '20
  IF @@ROWCOUNT = 0 BREAK;
END;
```



Optimized Locking

- Transaction ID (TID) locks replace long-held row and page locks
- Only lock as long as actually needed
- Lock After Qualification (LAQ): check the predicate first, lock only what qualifies
- Less blocking, less lock memory, no escalation

Optimized Locking: Who Flips the Switch

- Off by default, set per database
- Accelerated Database Recovery (ADR) must be enabled first
- LAQ only works with Read Commited Snapshot Isolation (RCSI)
- HOLDLOCK and REPEATABLE READ still hold locks to end of transaction
- Patch to CU9 first

Demo



Chapter 3

“I Didn’t Change Anything, I Swear”

It’s magic!!!



What Your ORM Says Behind Your Back

Pre-2025

- Problem: ORM compile storms and duplicate plans
- Many sessions compile the same statement at once
  - Stored Procs use a compile lock to prevent this
  - Most common on a cold cache

```
// C# EF Core 10
Var totals = await db.Orders.Where(o => o.CustomerID == id)
        .Select(o => o.Amount).ToListAsync();

-- SQL Server receives RPC call to sp_executesql
EXEC sp_executesql N'SELECT * FROM dbo.Orders WHERE CustomerID = @Id',
                   N'@Id int', @Id = 42;
```

Optimized sp_executesql

- Identical batches now wait on a compile lock
- First session compiles, the rest reuse the exec plan
- Off by default

```
ALTER DATABASE SCOPED CONFIGURATION
        SET OPTIMIZED_SP_EXECUTESQL = ON;
```

Constantly Spilling into TempDB?

- Your DBA can now cap TempDB use per workload w. Resource Governor
- Hit your limit – Error 1138 emitted
- Includes temp tables, table variables, spills and spools



A Few More Gems

- Batch mode enabled for more functions — UPGRADE
  - Math functions
  - DATETRUNC
- Degrees of Parallelism (DOP) feedback — COMPAT 160
  - Introduced in 2022, on by default in 2025
- Cardinality Estimation (CE) feedback for expressions
  - New in 2025, on by default
  - Learns from chunks of a query – different from 2022

How CE Feedback Fixes an Estimate

Pre-2025

- Watches queries that repeat with a bad estimate
- Tries a different assumption, via a Query Store hint
- Keeps it only if the plan changes and CPU drops

```
-- Two filters: estimate too low, too high
ASSUME_MIN_SELECTIVITY_FOR_FILTER_ESTIMATES
ASSUME_FULL_INDEPENDENCE_FOR_FILTER_ESTIMATES
-- Joins; TOP, EXISTS, FAST n
ASSUME_JOIN_PREDICATE_DEPENDS_ON_FILTERS
DISABLE_OPTIMIZER_ROWGOAL
```

2025: CE Feedback for Expressions

	CE feedback (2022)	For expressions (2025)
Learns from	One query that repeats	A filter or join, in any query
The fix	A Query Store hint on that one query	Applied at compile to any query with the same shape
Judged by	CPU time	The row estimate
In the plan	Hint source = CE feedback	`<CardinalityFeedback>`

Chapter 4

“How Do I Know It Worked?”

Proving it, and a few gems



The Extended Event That Ate the Weekend

- New MAX_DURATION option
  - Seconds, minutes, hours, or days
- Session stops itself
- Restarting the XE resets the clock

```
CREATE EVENT SESSION CatchTheSlowQuery ON SERVER
ADD EVENT sqlserver.sql_statement_completed
ADD TARGET package0.event_file (SET filename = N'CatchTheSlowQuery')
WITH (MAX_DURATION = 30 MINUTES);
```

New Extended Events!

- OPPO
  - `optional_parameter_optimization_skipped_reason`
- Optimized locking
  - `lock_after_qual_stmt_abort, locking_stats`
- CE feedback
  - `query_adhoc_ce_feedback_hint`
- Tempdb limits
  - `tempdb_data_workload_group_limit_reached`

Easier External Access

- Call an HTTPS endpoint from T-SQL
- Enable with sp_configure
- Needs EXECUTE ANY EXTERNAL ENDPOINT permission
- *Pre-2025:*
  - OLE Automation Procedures (sp_OA*), SQLCLR, & xp_cmdshell




```
EXEC sp_invoke_external_rest_endpoint
     @url = N'https://api.example.com/status',
     @method = 'GET', @response = @r OUTPUT;
```

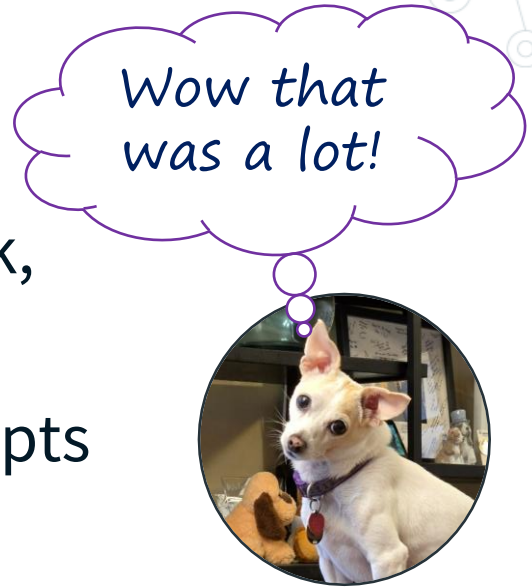
Meanwhile, on Azure SQL

- Automatic index compaction (GA, September 2026)
- Local time zone support (preview)
- Regex-based dynamic data masking (preview)

Review Complete

Approved, with comments

- **Replace:** regex hacks, math tricks, encoding tricks
- **Rewrite:** JSON storage, search procs, blocking workarounds
- **Leave alone:** sp_executesql, feedback, batch mode
- **Prove it:** time-bound XE and the receipts



What to Ask Your DBA For

UPGRADE	Up to the latest current CU *(CU10 as of 10/31/26)*
COMPAT 170	Compat level 170, with Query Store on
SWITCH	ADR and RCSI, then optimized locking
SWITCH	OPTIMIZED_SP_EXECUTESQL
PREVIEW	PREVIEW_FEATURES in dev only

Parting Thought



Practical *features don't always get the spotlight.*

Be careful not to miss them

Enhancing the industry proven engine

Security

- Security cache improvements
- OAEP support for encryption
- PBKDF password hashing
- Authentication using system-assigned managed identity
- Backup to URL with managed identity
- Managed identity support for EKM
- Managed identity for AI models
- Entra logins with nonunique display names
- Custom password policy on Linux
- TDS 8.0/TLS 1.3 support for tools and features

Performance

- Optimized Locking
- Tempdb space resource governance
- ADR in tempdb
- Persisted stats for readable secondaries
- Change tracking cleanup
- Columnstore index maintenance
- CE feedback for expressions
- Optional parameter plans optimization
- DOP feedback on by default
- Query store for readable secondaries
- ABORT_QUERY_EXECUTION query hint
- Optimized sp_executesql
- Batch mode optimizations
- Remove In-Memory OLTP from a database
- tmpfs support for tempdb in Linux

HADR

- Fast failover for persistent AG Health
- Async page request dispatching
- Improved health diagnostics
- Communication control flow tuning
- Switching to resolving state
- Remove listener IP address
- NONE for routing
- AG group commit waiting tuning
- Contained AG support for DAG
- DAG sync improvements
- Backups on secondary replicas
- ZSTD Backup compression
- Backup to Azure immutable storage

Resources

SQL Server 2025 Unveiled: Bob Ward

Apress (book)

Update: SQL Server 2025's REGEX Performance Isn't So Bad!: Brent Ozar

https://www.brentozar.com/archive/2025/12/update-sql-server-2025s-regex-performance-isnt-so-bad/

How SQL Server 2025's Optional Parameter Plan Optimization Works: Brent Ozar

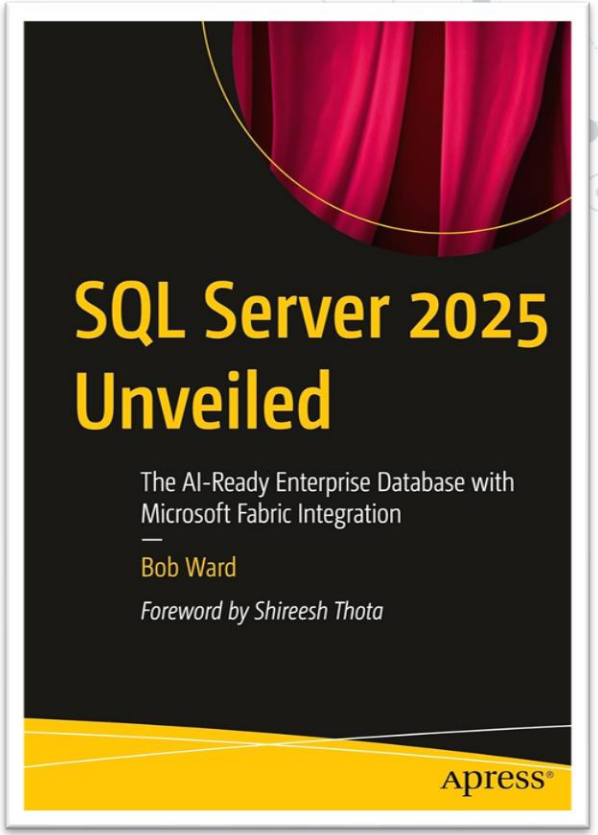
https://www.brentozar.com/archive/2025/05/how-sql-server-2025s-optional-parameter-plan-optimization-works/

What's New in SQL Server 2025: Microsoft Learn

https://learn.microsoft.com/en-us/sql/sql-server/what-s-new-in-sql-server-2025?view=sql-server-ver17

Cardinality Feedback Estimation Explained: Kate Smith

https://techcommunity.microsoft.com/blog/azuresqlblog/cardinality-estimation-feedback-explained-by-kate-smith-akatesmith/4197930



Thank you



Andy Yun

sqlbek@gmail.com
https://sqlbek.wordpress.com
https://github.com/sqlbek

Keep in Touch:

SQL Slack Community
https://dbatools.io/slack/

Reddit
r/SQLServer

Evaluate this session at:

Your feedback is important to us

passdatacommunitysummit.com/evaluations